

模拟卷 08-2025 风格题目与详解

目录

1	编译原理 2025 风格模拟卷 08（题目与详解）	2
2	A 卷题目	2
3	B 详解	4
4	C 复盘清单	9

1 编译原理 2025 风格模拟卷 08 (题目与详解)

1.1 卷面说明

本卷按 Slides/Slides/handout 与三份 Recap PPT 更新, 主线覆盖前端自动机与 LL、中端 IR/SSA、数据流、符号执行、后端目标代码、寄存器分配、指令调度以及过程间/指针分析。总分 100 分。

2 A 卷题目

2.1 一、词法分析 (15 分)

正则表达式:

$a^* b (a|b)^*$

1. 使用 Thompson 构造法构造等价 NFA。
2. 使用子集构造法构造 DFA, 写明 DFA 状态含义。
3. 最小化 DFA。

2.2 二、LL 语法分析 (15 分)

文法:

```
K -> L M
L -> L hash | key
M -> open | close
```

1. 消除左递归。
2. 写 FIRST 和 FOLLOW。
3. 写预测分析表。
4. 判断消除左递归后的文法是否为 LL(1)。

2.3 三、指令调度 (15 分)

给定指令:

```

I1: R1 = R2 + R3
I2: R4 = R1 + R5
I3: R2 = R6 + R7
I4: R1 = R8 + R9
I5: R10 = R4 + R1

```

1. 写出指令调度的目的。
2. 写出 RAW、WAR、WAW 的定义。
3. 构建依赖图，WAR/WAW 用虚线表示。
4. 在允许寄存器重命名的条件下给出一个合法重排序列。

2.4 四、支配问题（15 分）

CFG：循环带出口分支

```

Entry->H
H->Body
Body->If
If->Body
If->Exit

```

1. 写出 Dom、PostDom、Dom Frontier 的定义。
2. 计算每个节点的 Dom 集和 idom。
3. 写出支配边界。
4. 说明 SSA 中 phi 函数应放在什么位置。

2.5 五、符号执行与 SAT（20 分）

公式：

$$F = ((a8 \text{ and } b8) \text{ or } c8)$$

1. 写出 Path-sensitive、Flow-sensitive、Context-sensitive 的定义。
2. 写出 CNF 的定义。
3. 使用 Tseitin 算法把 F 转成 CNF。
4. 证明 2025 卷中的两组公式同时可满足。

2.6 六、Andersen 指针分析（20 分）

程序：

```
a = &01
b = &02
c = a
d = b
*c = d
e = *a
```

1. 对每条语句写 Andersen 约束。
2. 根据约束构建闭包。
3. 写出最终 points-to 集合。

2.7 七、PPT 新增重点：三地址码与控制流（10 分）

源程序：

```
while (a < b) {
    a = a + 1;
}
```

1. 写出一种三地址码。
2. 标出循环测试、循环体和出口标签。
3. 说明条件跳转和无条件跳转分别在哪里使用。

3 B 详解

3.1 一、词法分析详解

正则表达式： $a^*b(a|b)^*$

Thompson NFA: 起点 q_0 , 终点 q_7 。按下列转移表画图, q_7 画双圈。

```
q2 --a--> q3
q0 --epsilon--> q2
q0 --epsilon--> q1
q3 --epsilon--> q2
q3 --epsilon--> q1
q4 --b--> q5
q1 --epsilon--> q4
q10 --a--> q11
```

```

q12 --b--> q13
q8 --epsilon--> q10
q8 --epsilon--> q12
q11 --epsilon--> q9
q13 --epsilon--> q9
q6 --epsilon--> q8
q6 --epsilon--> q7
q9 --epsilon--> q8
q9 --epsilon--> q7
q5 --epsilon--> q6

```

子集构造 DFA:

D0:

```

NFA集合 = {q0,q1,q2,q4}
on a -> D1
on b -> D2
终态 = 否

```

D1:

```

NFA集合 = {q1,q2,q3,q4}
on a -> D1
on b -> D2
终态 = 否

```

D2:

```

NFA集合 = {q5,q6,q7,q8,q10,q12}
on a -> D3
on b -> D4
终态 = 是

```

D3:

```

NFA集合 = {q7,q8,q9,q10,q11,q12}
on a -> D3
on b -> D4
终态 = 是

```

D4:

```

NFA集合 = {q7,q8,q9,q10,q12,q13}
on a -> D3
on b -> D4
终态 = 是

```

最小化:

初分组:

```

终态组 = {D2,D3,D4}
非终态组 = {D0,D1}

```

最终分组:

```

G1 = {D2,D4,D3}
G2 = {D0,D1}

```

每个最终分组对应一个最小 DFA 状态。

3.2 二、LL 语法分析详解

原文法:

$K \rightarrow L M$

```
L -> L hash | key
M -> open | close
```

消除左递归:

```
K -> L M
L -> key L'
L' -> hash L' | epsilon
M -> open | close
```

FIRST:

```
FIRST(K)={ key }
FIRST(L)={ key }
FIRST(L')={ hash, epsilon }
FIRST(M)={ open, close }
```

FOLLOW:

```
FOLLOW(K)={ $ }
FOLLOW(L)={ open, close }
FOLLOW(L')={ open, close }
FOLLOW(M)={ $ }
```

预测表:

```
M[K,key] = K -> L M
M[L,key] = L -> key L'
M[L',hash] = L' -> hash L'
M[L',open] = L' -> epsilon
M[L',close] = L' -> epsilon
M[M,open] = M -> open
M[M,close] = M -> close
```

结论: 表中无多重入口, 消除左递归后的文法是 LL(1)。

3.3 三、指令调度详解

定义:

指令调度在不改变程序语义的前提下重排指令, 减少流水线停顿, 提高并行执行效率。

RAW: 先写后读, 必须保持顺序。

WAR: 先读后写, 写者不能提前。

WAW: 先写后写, 顺序影响最终值。

指令:

```
I1: R1 = R2 + R3
I2: R4 = R1 + R5
I3: R2 = R6 + R7
I4: R1 = R8 + R9
I5: R10 = R4 + R1
```

依赖:

```
I1 -> I2 RAW on R1
```

```

I1 -> I3 WAR on R2
I1 -> I4 WAW on R1
I2 -> I4 WAR on R1
I2 -> I5 RAW on R4
I4 -> I5 RAW on R1

```

寄存器重命名:

```

I3: R2_new = R6 + R7
I4: R1_new = R8 + R9
I5 使用 R1_new

```

重命名后保留 RAW, WAR/WAW 被消除。一个合法调度序列:

```

I1, I3, I4, I2, I5

```

3.4 四、支配问题详解

定义:

Dom(n): 入口到 n 的每条路径都经过的节点集合。

PostDom(n): n 到出口的每条路径都经过的节点集合。

Dom Frontier(n): n 支配某个前驱, 但不严格支配该节点的位置。

CFG 边:

```

Entry->H
H->Body
Body->If
If->Body
If->Exit

```

Dom 集:

```

Dom(Entry)={Entry}
Dom(H)={Entry,H}
Dom(Body)={Entry,H,Body}
Dom(If)={Entry,H,Body,If}
Dom(Exit)={Entry,H,Body,If,Exit}

```

直接支配者:

```

idom(H)=Entry
idom(Body)=H
idom(If)=Body
idom(Exit)=If

```

支配边界:

```

DF(Body)={Body}
DF(If)={Body}

```

其余节点 DF 为空集

SSA phi 放置: 若变量在多个基本块中被定义, 则在这些定义块的迭代支配边界处插入该变量的 phi。直观上, phi 放在多

3.5 五、符号执行与 SAT 详解

公式: $F = ((a8 \text{ and } b8) \text{ or } c8)$

引入: $x81 \leftrightarrow (a8 \text{ and } b8)$, $x82 \leftrightarrow (x81 \text{ or } c8)$, 根变量 $x82$ 为真。

CNF:

```
(not x81 or a8)
(not x81 or b8)
(x81 or not a8 or not b8)
(x82 or not x81)
(x82 or not c8)
(not x82 or x81 or c8)
(x82)
```

敏感性定义:

Path-sensitive 区分不同路径。

Flow-sensitive 区分程序点和语句顺序。

Context-sensitive 区分函数调用上下文。

同时可满足证明套用:

$F1 = (a0 \text{ or } \dots \text{ or } an \text{ or } c)$ and $(b0 \text{ or } \dots \text{ or } bm \text{ or } \text{not } c)$, $F2 = (a0 \text{ or } \dots \text{ or } an \text{ or } b0 \text{ or } \dots \text{ or } bm)$ 。

$F1$ 为真时, $c = \text{false}$ 推出某个 $ai = \text{true}$, $c = \text{true}$ 推出某个 $bj = \text{true}$, 所以 $F2$ 为真。

$F2$ 为真时, 若某个 $ai = \text{true}$ 令 $c = \text{false}$; 若某个 $bj = \text{true}$ 令 $c = \text{true}$, 所以 $F1$ 为真。

3.6 六、Andersen 指针分析详解

程序:

```
a = &01
b = &02
c = a
d = b
*c = d
e = *a
```

约束:

```
01 in pts(a)
02 in pts(b)
pts(a) subset pts(c)
pts(b) subset pts(d)
for v in pts(c), pts(d) subset pts(v)
for v in pts(a), pts(v) subset pts(e)
```

闭包:

```
pts(a)={01}
pts(b)={02}
pts(c)={01}
pts(d)={02}
由 *c=d 得 pts(01)={02}
由 e=*a 得 pts(e)={02}
```


3.7 七、PPT 新增重点详解

```
L1:
    if a < b goto L2
    goto L3
L2:
    t1 = a + 1
    a = t1
    goto L1
L3:
```

L1 是循环测试。

L2 是循环体入口。

L3 是循环出口。

条件跳转用于根据 $a < b$ 选择进入循环体或退出。

无条件跳转 `goto L1` 用于循环体结束后回到测试位置。

4 C 复盘清单

词法: Thompson、epsilon-closure、move、终态集合、最小化分组。

LL: 左递归、FIRST、FOLLOW、预测表、多重入口。

IR/SSA: 三地址码、基本块、支配边界、phi 插入与转出。

数据流: 方向、meet、transfer、gen/kill、worklist、IN/OUT。

后端: 目标代码、地址模式、冲突图、spill、指令调度。

符号执行: 敏感性、CNF、Tseitin 子句、同时可满足证明。

过程间/指针: 调用上下文、上下文敏感、Andersen 约束与闭包。